

UNLOCKING THE POWER OF LINUX

A COMPREHENSIVE GUIDE TO CUSTOM SERVICES
AND SCHEDULING

Prepared By: Hosni Zaaraoui

Linux is the backbone of modern computing, powering everything from servers to embedded systems. One of its most powerful features is the ability to create custom services and automate tasks using scheduling tools. Whether you're a system administrator, DevOps engineer, or a curious developer, understanding how to create and manage custom services is essential for optimizing workflows and ensuring system reliability.

In this guide, we'll explore:

- How to create custom Linux services using systemd and init.d.
- Scheduling methods like cron, systemd timers, and the at command.
- Best practices for managing and securing your services.
- Real-world examples to help you apply these concepts.

Let's dive in!

Building Custom Linux Services: From Basics to Best Practices

What is a Linux Service?

A Linux service is a background process that runs independently of user interaction. Services are typically used to manage long-running applications like web servers, databases, or custom scripts. They can be started, stopped, and monitored using system tools.

Using systemd

systemd is the default init system for most modern Linux distributions. It uses unit files to define and manage services.

– Steps to Create a systemd Service:

1. Create a Service Unit File:

- Save the file in `/etc/systemd/system/`.
- **Example:** `/etc/systemd/system/my-custom-service.service`

```
[Unit]
Description=My Custom Service
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/my-script.py
Restart=always
User=nobody
Group=nogroup

[Install]
WantedBy=multi-user.target
return go(f, seed, [])
}
```

2. Reload systemd:



```
sudo systemctl daemon-reload
```



```
sudo systemctl start my-custom-service  
sudo systemctl enable my-custom-service
```

3. Check the Status:



```
sudo systemctl status my-custom-service
```

Using init.d (Legacy Systems)

For older systems that use SysVinit, you can create custom services using init scripts.

– Steps to Create an init.d Script:

1. Create a Script:

➤ Save the file in /etc/init.d/.

➤ **Example:** /etc/init.d/my-custom-service

```
#!/bin/bash
# Description: My Custom Service

case "$1" in
    start)
        /usr/bin/python3 /opt/my-script.py &
        ;;
    stop)
        killall my-script.py
        ;;
    *)
        echo "Usage: /etc/init.d/my-custom-service {start|stop}"
        exit 1
        ;;
esac
exit 0
```

2. Make the Script Executable:

```
sudo chmod +x /etc/init.d/my-custom-service
```

3. Start the Service:



```
sudo /etc/init.d/my-custom-service start
```

Best Practices

– Logging:

- ✓ Use journalctl or redirect output to a log file.



```
[Service]  
ExecStart=/usr/bin/python3 /opt/my-script.py >> /var/log/my-service.log 2>&1
```

– Security:

- ✓ Run services as non-root users.

– Testing:

- ✓ Test services in a staging environment before deploying to production.

Automating Tasks: Cron, systemd Timers, and More

Cron Jobs

– What is Cron?

Cron is a time-based job scheduler in Unix-like operating systems (including Linux). It allows you to automate repetitive tasks by scheduling commands or scripts to run at specific times or intervals. Cron is incredibly versatile and is widely used for tasks like backups, system maintenance, log rotation, and more.

– How Cron Works

Cron Daemon:

The cron daemon (background process) runs continuously and checks for scheduled tasks.

It reads the crontab (cron table) files to determine which tasks to run and when.

Crontab Files:

Each user can have their own crontab file, and there's also a system-wide crontab.

Crontab files contain lines of commands, each with a schedule and the command to execute.

Cron Jobs:

A cron job is a single task defined in a crontab file.

Each cron job specifies when to run and what to run.

Cron Syntax

A cron job is defined by a line in the crontab file with the following structure:



```
* * * * * command-to-execute
```

- The five asterisks (*) represent the schedule in this order:

Field	Allowed Values	Description
Minute	0–59	The minute the task should run.
Hour	0–23	The hour the task should run.
Day of Month	1–31	The day of the month the task should run.
Month	1–12 (or names like Jan)	The month the task should run.
Day of Week	0–7 (0 and 7 = Sunday)	The day of the week the task should run.

– Special Characters in Cron

Cron supports special characters to make scheduling more flexible:

Character	Description
*	Matches all values (e.g., * in the hour field means "every hour").
,	Specifies a list of values (e.g., 1,3,5 in the day field means "1st, 3rd, and 5th").
-	Specifies a range of values (e.g., 1-5 in the day field means "1st to 5th").
/	Specifies intervals (e.g., */2 in the hour field means "every 2 hours").

– Examples of Cron Jobs

Here are some practical examples to illustrate how cron works:

Run a Script Every Day at 2 AM:



```
0 2 * * * /usr/bin/python3 /opt/my-script.py
```

- 0: Minute 0 (start of the hour).
- 2: Hour 2 (2 AM).
- * * *: Every day, every month, every day of the week.

Run a Backup Script Every Sunday at Midnight:



```
0 0 * * 0 /opt/backup-script.sh
```

- 0 0: Midnight.
- * *: Every day of the month, every month.
- 0: Sunday.

Run a Log Cleanup Script Every 15 Minutes:



```
*/15 * * * * /opt/cleanup-logs.sh
```

- */15: Every 15 minutes.
- * * * *: Every hour, every day, every month, every day of the week.

Run a Script at 8 AM on Weekdays (Monday to Friday):



```
0 8 * * 1-5 /opt/daily-report.sh
```

- 0 8: 8 AM.
- * *: Every day of the month, every month.
- 1-5: Monday to Friday.

Run a Script on the 1st of Every Month at 6 AM:



```
0 6 1 * * /opt/monthly-report.sh
```

- 0 6: 6 AM.
- 1: 1st day of the month.
- * *: Every month, every day of the week.

– Managing Crontab Files

Edit Your Crontab:

To create or edit your crontab file, use the `crontab -e` command:



```
crontab -e
```

List Your Cron Jobs:

To view your current cron jobs, use the `crontab -l` command:



```
crontab -l
```

Remove All Cron Jobs:

To delete all cron jobs for the current user, use the `crontab -r` command:



```
crontab -r
```

System-Wide Crontab:

For system-wide cron jobs, edit the `/etc/crontab` file or place scripts in `/etc/cron.d/`.

– Cron Logs

Cron logs all executed jobs, making it easy to debug or verify that tasks are running as expected. On most systems, cron logs are stored in `/var/log/syslog` or `/var/log/cron`.

To view cron logs:



```
grep CRON /var/log/syslog
```

– When to Use Cron

Cron is ideal for:

- **Recurring Tasks:** Tasks that need to run at regular intervals (e.g., backups, log rotation).
- **Scheduled Maintenance:** Tasks like system updates or cleanup scripts.
- **Automation:** Automating repetitive tasks to save time and reduce human error.

– Best Practices for Using Cron

Use Absolute Paths:

Always use absolute paths for commands and scripts in cron jobs. Cron runs with a limited environment, so relative paths may not work.

Redirect Output:

Redirect the output of cron jobs to a log file for debugging:



```
* * * * * /opt/my-script.sh >> /var/log/my-script.log 2>&1
```

Test Your Cron Jobs:

Test your cron jobs manually before scheduling them to ensure they work as expected.

Avoid Overlapping Jobs:

If a cron job takes longer to run than the interval between executions, it can cause overlaps. Use locking mechanisms (e.g., flock) to prevent this.

systemd Timers

– What are systemd Timers?

systemd timers are a modern alternative to cron jobs for scheduling tasks in Linux systems that use systemd as their init system. Unlike cron, which is time-based, systemd timers are event-driven and offer greater flexibility, precision, and integration with the rest of the system.

– Key Features of systemd Timers

Event-Based Scheduling:

Timers can be triggered by calendar events (like cron) or by other system events (e.g., after boot, after another service starts).

Persistent Scheduling:

If a system is offline when a timer is supposed to trigger, systemd can catch up and run the missed task when the system comes back online.

Dependency Management:

Timers can depend on other services or units, ensuring tasks run only when specific conditions are met.

Logging and Monitoring:

Timers integrate with journalctl, making it easy to monitor and debug scheduled tasks.

Flexible Time Specifications:

Timers support more complex scheduling than cron, including intervals like "every 2 hours" or "every 15 minutes."

– How systemd Timers Work

A systemd timer consists of two files:

- **Timer Unit File:** Defines when the task should run.
- **Service Unit File:** Defines what task to run.

– Creating a systemd Timer

Step 1: Create a Service Unit File

The service unit file defines the task to be executed. For example, let's create a service to run a backup script.

File: /etc/systemd/system/backup.service



```
[Unit]
Description=Backup Service

[Service]
ExecStart=/opt/backup-script.sh
```

Step 2: Create a Timer Unit File

The timer unit file defines when the service should run.

File: /etc/systemd/system/backup.timer



```
[Unit]
Description=Run Backup Daily

[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

Step 3: Enable and Start the Timer

Reload systemd to recognize the new units:



```
sudo systemctl daemon-reload
```

Start and enable the timer:



```
sudo systemctl start backup.timer
sudo systemctl enable backup.timer daemon-reload
```

Check the status of the timer:



```
sudo systemctl status backup.timer
```

– Timer Scheduling Options

Systemd timers offer several ways to define when a task should run:

1. Calendar-Based Scheduling

Use the OnCalendar directive to specify a time or interval.

Examples:

Every day at 3 AM:



```
OnCalendar=*-*-* 03:00:00
```

Every Monday at 8 AM:



```
OnCalendar=Mon *-*-* 08:00:00
```

Every 15 minutes:



```
OnCalendar=*-*-* *:0/15:00
```

2. Monotonic Timers

These timers are based on events like system startup or service activation.

Examples:

Run 15 minutes after boot:



```
OnBootSec=15min
```

Run 1 hour after the last activation:



```
OnUnitActiveSec=1h
```

3. Persistent Timers

If the system is offline when a timer is supposed to trigger, the task will run as soon as the system comes back online.



```
Persistent=true
```

– Advantages of systemd Timers Over Cron

Flexibility:

Timers can be triggered by calendar events, system events, or intervals.

They support more complex scheduling than cron.

Integration with systemd:

Timers can depend on other services or units, ensuring tasks run only when specific conditions are met.

Logging and Monitoring:

Timers integrate with journalctl, making it easy to monitor and debug scheduled tasks.

Persistent Scheduling:

If a system is offline when a timer is supposed to trigger, systemd can catch up and run the missed task when the system comes back online.

Dependency Management:

Timers can depend on other services or units, ensuring tasks run only when specific conditions are met.

at Command

The at command is a powerful yet often overlooked tool for scheduling one-time tasks in Linux. Unlike cron, which is designed for recurring tasks, at is perfect for running a command or script at a specific time in the future.

– How the at Command Works

Syntax:



```
echo "command" | at [time]
```

- command: The command or script you want to run.
- time: The time when the command should be executed.

Time Formats:

- Absolute Time: at 10:00 PM, at 14:30, at midnight.
- Relative Time: at now + 1 hour, at now + 30 minutes, at tomorrow.

Examples:

✓ Run a script at 10:00 PM:



```
echo "/usr/bin/python3 /opt/my-script.py" | at 10:00 PM
```

✓ Run a command in 2 hours:



```
echo "tar -czf /backups/backup-$(date +%Y%m%d).tar.gz /var/www/html" | at  
now + 2 hours
```

✓ Schedule a shutdown at midnight:



```
echo "shutdown -h now" | at midnight
```

– View Scheduled Jobs:

Use the atq command to list pending jobs:



```
atq
```

– Remove a Scheduled Job:

Use the atrm command followed by the job number:



```
atrm 1
```

– Logs and Output:

By default, the output of at jobs is emailed to the user who scheduled the job.

To redirect output to a file:



```
echo "/usr/bin/python3 /opt/my-script.py > /var/log/my-script.log 2>&1" | at  
now + 1 hour
```

– When to Use at

- One-Time Tasks: Perfect for tasks that don't need to recur, like running a backup before a system update.

- Ad-Hoc Scheduling: Great for quick, on-the-fly scheduling without editing crontab.
- Testing: Useful for testing scripts or commands at a specific time without setting up a cron job.

Practical Applications: Ready-to-Use Scripts

Example 1: Disk Usage Monitor

Objective: Create a service to monitor disk usage and send alerts.

Script: /opt/disk-usage-monitor.py

```
#!/usr/bin/python3
import shutil
import smtplib
from email.mime.text import MIMEText

# Disk usage threshold (in percentage)
THRESHOLD = 80

# Email configuration
EMAIL_FROM = "alert@example.com"
EMAIL_TO = "admin@example.com"
SMTP_SERVER = "smtp.example.com"

# Check disk usage
usage = shutil.disk_usage("/")
percent_used = (usage.used / usage.total) * 100

if percent_used > THRESHOLD:
    # Send email alert
    msg = MIMEText(f"Disk usage is at {percent_used:.2f}%!")
    msg["Subject"] = "Disk Usage Alert"
    msg["From"] = EMAIL_FROM
    msg["To"] = EMAIL_TO

    with smtplib.SMTP(SMTP_SERVER) as server:
        server.sendmail(EMAIL_FROM, EMAIL_TO, msg.as_string())
```

Service File: /etc/systemd/system/disk-monitor.service



```
[Unit]
Description=Disk Usage Monitor

[Service]
ExecStart=/usr/bin/python3 /opt/disk-usage-monitor.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Cron Job: Run the script hourly.



```
0 * * * * /usr/bin/python3 /opt/disk-usage-monitor.py
```

Example 2: Automated Backups

Objective: Use a systemd timer to run a backup script daily.

Script: /opt/backup-script.sh



```
#!/bin/bash
BACKUP_DIR="/backups"
SOURCE_DIR="/var/www/html"
TIMESTAMP=$(date +"%Y%m%d%H%M%S")

# Create backup
tar -czf "$BACKUP_DIR/backup-$TIMESTAMP.tar.gz" "$SOURCE_DIR"

# Delete backups older than 7 days
find "$BACKUP_DIR" -type f -mtime +7 -exec rm {} \;
```

Service File: /etc/systemd/system/backup.service



```
[Unit]
Description=Daily Backup Service

[Service]
ExecStart=/opt/backup-script.sh
```

Timer File: /etc/systemd/system/backup.timer



```
[Unit]
Description=Run Backup Daily

[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

Example 3: Web Application Service

Objective: Create a systemd service to run a Python web app.

Script: /opt/web-app.py



```
#!/usr/bin/python3
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, World!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Service File: /etc/systemd/system/web-app.service



```
[Unit]
Description=Python Web Application

[Service]
ExecStart=/usr/bin/python3 /opt/web-app.py
Restart=always
User=www-data
Group=www-data

[Install]
WantedBy=multi-user.target
```

Wrapping Up: Your Path to Linux Mastery

By now, you've learned how to create custom Linux services, automate tasks with scheduling tools, and apply these skills to real-world scenarios. Whether you're managing a single server or a large-scale infrastructure, these techniques will help you optimize your workflows and ensure system reliability.

Feel free to experiment with the examples provided and adapt them to your own use cases. If you found this guide helpful, share it with your network and let's continue the conversation in the comments!